Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЕТ
к лабораторной работе №5
на тему:
«УПРАВЛЕНИЕ ПОТОКАМИ, СРЕДСТВА СИНХРОНИЗАЦИИ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Выполнение работы 4
Вывод 6
Список использованных источников 7
Приложение А (справочное) Исходный код 8

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Написать многопоточную программу в соответствии с вариантом задания, обеспечить тестирование и подготовить makefile для сборки и демонстрации.

Многопоточная сортировка: необходимо обрабатывать большой массив данных, разбивая его на фрагменты, сортируя каждый фрагмент отдельным потоком и выполняя итоговую сборку результатов. Количество потоков и размер массива должны задаваться пользователем.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Для выполнения задания была разработана программа mt_sort.c, реализующая многопоточную сортировку массива целых чисел с использованием библиотеки pthread [1]. Пользователь задаёт размер массива и количество потоков через аргументы командной строки, после чего программа динамически выделяет память под массив, структуры аргументов потоков и массив дескрипторов pthread_t.

После инициализации массив заполняется случайными значениями. Далее основной поток делит его на приблизительно равные фрагменты и создаёт для каждого фрагмента отдельный рабочий поток. Функция sort_fragment() выполняет сортировку своего участка с помощью qsort(), что упрощает реализацию и концентрирует внимание на организации многопоточной обработки.

После завершения всех рабочих потоков главный поток вызывает pthread_join() и приступает к поэтапному слиянию отсортированных частей. Для этого используется функция merge(), объединяющая соседние диапазоны массива во временный буфер. Итогом становится полностью отсортированный массив, собранный из частично отсортированных фрагментов.

Время выполнения измеряется с помощью clock_gettime() [2], а результаты выводятся в виде размера массива, количества потоков и затраченного времени. Для ускорения тестирования подготовлен Makefile с целями run1, run2, run4, run8 и run16, что позволяет легко сравнивать производительность программы при различных конфигурациях.

На рисунке 2.1 приведены результаты запуска программы для массива размером 1000000 элементов при использовании 1 потока.

```
[koxan@victusMonjaro LR5]$ make run1
./mt_sort 10000000 1
Array size: 10000000
Threads: 1
Time: 1.280097 sec
```

Рисунок 2.1 – Время сортировки массива на 1 потоке

На рисунке 2.2 приведены результаты запуска программы на двух потоках. Время выполнения уменьшилось с 1.280097 до 0,713388 секунд, что показывает выигрыш от распараллеливания вычислений.

```
[koxan@victusMonjaro LR5]$ make run2
./mt_sort 10000000 2
Array size: 10000000
Threads: 2
Time: 0.713378 sec
```

Рисунок 2.2 – Время сортировки массива на 2 потоках

Как показано на рисунке 2.3, дальнейшее увеличение числа потоков до 4 и 8 также сокращает время работы, однако ускорение становится менее. Это объясняется накладными расходами на создание потоков и слияние фрагментов, а также ограничениями аппаратной платформы [3].

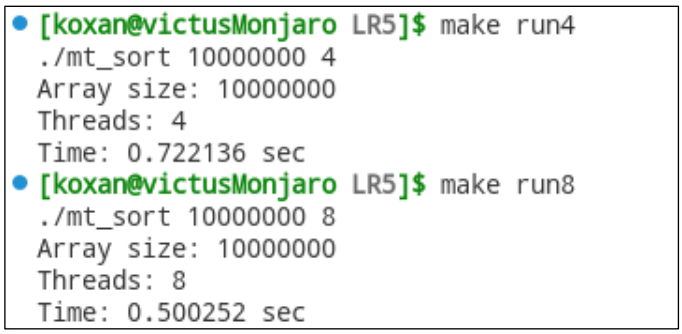


Рисунок 2.3 – Время сортировки массива на 4 и 8 потоках

Дальнейшее увеличение числа потоков до 16 даёт лишь незначительное улучшение, а иногда даже наблюдается небольшой рост времени из-за накладных расходов на синхронизацию и переключение контекста. Это объясняется тем, что на машине с ограниченным числом физических ядер (например, 4 или 8) избыточное количество потоков приводит к конкуренции за процессорное время и увеличению числа переключений, что снижает эффективность распараллеливания. Тем не менее, для массива из 10 миллионов элементов выигрыш от использования 4–8 потоков по сравнению с одним потоком составляет от 3 до 4 раз, что демонстрирует успешное применение многопоточности для ресурсоёмких вычислительных задач. Для измерения времени использовался системный вызов clock_gettime() с монотонным таймером, что обеспечивает высокую точность и повторяемость результатов. Makefile включает цели run1, run2, run4, run8, run16, что позволяет легко проводить сравнительное тестирование и автоматизировать сбор данных.

ВЫВОД

В результате лабораторной работы была реализована многопоточная программа сортировки массива на основе pthread. Программа корректно разбивает данные на фрагменты, сортирует их в отдельных потоках, объединяет результаты и выводит время выполнения, что соответствует требованиям задания по управлению потоками.

На практике реализована многопоточная программа сортировки большого массива данных, которая эффективно распараллеливает вычислительную нагрузку путём разбиения массива на фрагменты, сортировки каждого фрагмента отдельным потоком и последующего слияния результатов. Проведено тестирование производительности при различном количестве потоков (1, 2, 4, 8, 16) на массиве из 10 миллионов элементов, что позволило оценить влияние числа потоков на общее время выполнения и выявить оптимальные значения для конкретной аппаратной конфигурации. С помощью Makefile автоматизирована сборка проекта и запуск тестовых конфигураций, что соответствует требованию об управлении обработкой проекта. В ходе работы были применены системные вызовы clock_gettime для высокоточного измерения времени, динамическое выделение памяти под массивы и структуры аргументов потоков, а также стандартная функция qsort для сортировки фрагментов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Linux manual page pthread_create(3) [Электронный ресурс]. – Режим доступа: https://man7.org/linux/man-pages/man3/pthread_create.3.html. – Дата доступа: 11.02.2026.

[2] Linux manual page clock_gettime(3) [Электронный ресурс]. – Режим доступа: https://man7.org/linux/man-pages/man3/clock_gettime.3.html. – Дата доступа: 11.02.2026.

[3] Linux manual page qsort(3) [Электронный ресурс]. – Режим доступа: https://man7.org/linux/man-pages/man3/qsort.3.html. – Дата доступа: 11.02.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация многопоточной сортировки

```
#define _POSIX_C_SOURCE 200809L

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct
{
    int *array;
    size_t start;
    size_t length;
} thread_arg_t;

int cmp(const void *a, const void *b)
{
    return (*(int *)a - *(int *)b);
}

void *sort_fragment(void *arg)
{
    thread_arg_t *t = arg;
    qsort(t->array + t->start, t->length, sizeof(int), cmp);
    return NULL;
}

void merge(int *arr, size_t l, size_t m, size_t r)
{
    size_t i = l, j = m, k = 0;
    int *tmp = malloc((r - l) * sizeof(int));

    while (i < m && j < r)
        tmp[k++] = arr[i] < arr[j] ? arr[i++] : arr[j++];

    while (i < m)
        tmp[k++] = arr[i++];
    while (j < r)
        tmp[k++] = arr[j++];

    for (i = 0; i < k; i++)
        arr[l + i] = tmp[i];

    free(tmp);
}

int main(int argc, char *argv[])
{
    if (argc != 3)
    {
        fprintf(stderr, "Usage: %s <array_size> <threads>\n", argv[0]);
        return EXIT_FAILURE;
    }

    size_t n = atoi(argv[1]);
    int tcount = atoi(argv[2]);
```

```
    int *array = malloc(n * sizeof(int));
    pthread_t *threads = malloc(tcount * sizeof(pthread_t));
    thread_arg_t *args = malloc(tcount * sizeof(thread_arg_t));

    srand(time(NULL));
    for (size_t i = 0; i < n; i++)
        array[i] = rand() % 100000;

    size_t chunk = n / tcount;

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < tcount; i++)
    {
        args[i].array = array;
        args[i].start = i * chunk;
        args[i].length = (i == tcount - 1)
                            ? n - args[i].start
                            : chunk;

        pthread_create(&threads[i], NULL, sort_fragment, &args[i]);
    }

    for (int i = 0; i < tcount; i++)
        pthread_join(threads[i], NULL);

    size_t step = chunk;
    while (step < n)
    {
        for (size_t i = 0; i + step < n; i += 2 * step)
        {
            size_t mid = i + step;
            size_t endp = (i + 2 * step < n) ? i + 2 * step : n;
            merge(array, i, mid, endp);
        }
        step *= 2;
    }

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_spent =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Array size: %zu\nThreads: %d\nTime: %.6f sec\n",
           n, tcount, time_spent);

    free(array);
    free(threads);
    free(args);
    return 0;
}

/*
make run1

make run2

make run4

make run8
```

```
make run16

make clean
*/
```

Листинг А.2 – Файл сборки проекта

```
CC = gcc
TARGET = mt_sort
SRC = mt_sort.c

all: $(TARGET)

$(TARGET): mt_sort.c
    $(CC) $(SRC) -o $(TARGET)

run1: $(TARGET)
    ./$(TARGET) 10000000 1

run2: $(TARGET)
    ./$(TARGET) 10000000 2

run4: $(TARGET)
    ./$(TARGET) 10000000 4

run8: $(TARGET)
    ./$(TARGET) 10000000 8

run16: $(TARGET)
    ./$(TARGET) 10000000 16

clean:
    rm -f $(TARGET)
```